ENME 408 Spring 2026
Project 2

Ethan Hardesty
Christopher Appiah

Honor code:
"I have neither received nor given aid to anyone outside my team during this project."

1. **Linearization and attitude dynamics.**

   (a) Linearize the rotational equations of motion about the hover equilibrium $x_{\text{att}} = 0_{6\times1}$ and $u_{\text{att}} = 0_{3\times1}$ to obtain the linearized attitude dynamics in state-space form

$$\dot{x}_{\text{att}} = A_{\text{att}} x_{\text{att}} + B_{\text{att}} u_{\text{att}}.$$

The full nonlinear dynamics are given by:

$$m\ddot{p} = mge_3 + C_A D(\cdot) \tag{1}$$
$$\dot{q} = S(q)^{-1}\omega \tag{2}$$
$$J\dot{\omega} = \omega \times J\omega + \tau \tag{3}$$

where:

- $q = (\phi, \theta, \psi)$ are Euler angles,
- $\omega = (\omega_1, \omega_2, \omega_3)$ is angular velocity,
- J is the inertia matrix,
- $\tau = (\tau_1, \tau_2, \tau_3)$ is the control torque input.

Answer:

Equilibrium Point:
We linearize about the hover equilibrium:

$$x_{\text{att}} = 0_{6\times1}, \quad u_{\text{att}} = 0_{3\times1} \tag{4}$$

where the state and input vectors are defined as:

$$x_{\text{att}} = \begin{bmatrix}\phi & \theta & \psi & \omega_1 & \omega_2 & \omega_3\end{bmatrix}^T \tag{5}$$
$$u_{\text{att}} = \begin{bmatrix}\tau_1 & \tau_2 & \tau_3\end{bmatrix}^T \tag{6}$$

Kinematic Linearization:
From (2):

$$\dot{q} = S(q)^{-1}\omega \tag{7}$$

Near the equilibrium ($q \approx 0$), we approximate:

$$S(q)^{-1} \approx I \tag{8}$$

yielding:

$$\dot{\phi} = \omega_1 \tag{9}$$
$$\dot{\theta} = \omega_2 \tag{10}$$
$$\dot{\psi} = \omega_3 \tag{11}$$

Rotational Dynamics Linearization:

From (3):

$$J\dot{\omega} = \omega \times J\omega + \tau \tag{12}$$

At hover:

$$\omega = 0 \;\Rightarrow\; \omega \times J\omega = 0 \tag{13}$$

Therefore:

$$J\dot{\omega} = \tau \;\Rightarrow\; \dot{\omega} = J^{-1}\tau \tag{14}$$

Assuming a diagonal inertia matrix:

$$J = \begin{bmatrix} J_1 & 0 & 0 \\ 0 & J_2 & 0 \\ 0 & 0 & J_3 \end{bmatrix} \tag{15}$$

we obtain:

$$\dot{\omega}_1 = \frac{1}{J_1}\tau_1 \tag{16}$$

$$\dot{\omega}_2 = \frac{1}{J_2}\tau_2 \tag{17}$$

$$\dot{\omega}_3 = \frac{1}{J_3}\tau_3 \tag{18}$$

Linear State-Space Model

Combining the kinematics and rotational dynamics, we obtain:

$$\dot{x}_{\text{att}} = A_{\text{att}} x_{\text{att}} + B_{\text{att}} u_{\text{att}} \tag{19}$$

System Matrix

$$A_{\text{att}} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{20}$$

Input Matrix

$$B_{\text{att}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{1}{J_1} & 0 & 0 \\ 0 & \frac{1}{J_2} & 0 \\ 0 & 0 & \frac{1}{J_3} \end{bmatrix} \tag{21}$$

(b) Verify controllability of the linearized attitude subsystem.

Answer: The controllability matrix is defined as

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix} \tag{22}$$

The input matrix is

$$B = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{1}{J_1} & 0 & 0 \\ 0 & \frac{1}{J_2} & 0 \\ 0 & 0 & \frac{1}{J_3} \end{bmatrix}. \tag{23}$$

The state matrix is

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{24}$$

We compute

$$AB = \begin{bmatrix} \frac{1}{J_1} & 0 & 0 \\ 0 & \frac{1}{J_2} & 0 \\ 0 & 0 & \frac{1}{J_3} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad A^2B = 0. \tag{25}$$

Thus, the controllability matrix becomes

$$\mathcal{C} = \begin{bmatrix} 0 & 0 & 0 & \frac{1}{J_1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{J_2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{J_3} & 0 & 0 & 0 \\ \frac{1}{J_1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{J_2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{J_3} & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{26}$$

Since the columns are linearly independent,

$$\operatorname{rank}(\mathcal{C}) = 6.$$

Therefore, the linearized attitude subsystem is controllable.

2. **Full-state feedback attitude stabilization.**

   (a) Design a full-state feedback controller

$$u_{\text{att}} = K_{\text{att}} x_{\text{att}} \tag{27}$$

   using pole placement or LQR.

   (b) Simulate the closed-loop nonlinear attitude dynamics using the full nonlinear rotational equations of motion (EOM).

   (c) Use a set of nonzero initial conditions (e.g., $\phi(0)$, $\theta(0)$, $\psi(0)$, $\omega(0)$) to demonstrate stabilization to hover.

   (d) Plot Euler angles and angular velocities versus time, and comment on transient response (rise time, overshoot, settling time).

Answers a-d:

Using LQR to control the linearized system, we made the following Simulink model,

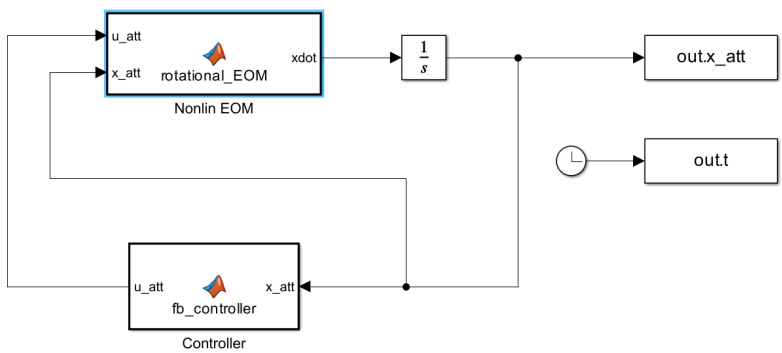

Figure 1. Full-State Feedback Simulink Controller with LQR

With the following LQR MATLAB code to find K_{att},

```
Q = 2*eye(6);
R = 10;

K_att = lqr(A_att, -B_att, Q, R);
```

Which uses the A_{att} and B_{att} matrices found in part 1. Using a set of randomly picked nonzero initial conditions for the vector x_{att} ($\phi_0 = 45°$, $\theta_0 = 5°$, $\psi_0 = 30$, $\omega_1 = 5$ rad/s, $\omega_2 = 10$ rad/s, $\omega_3 = 15$ rad/s) we can run the model and plot the euler angles and angular velocities versus time.

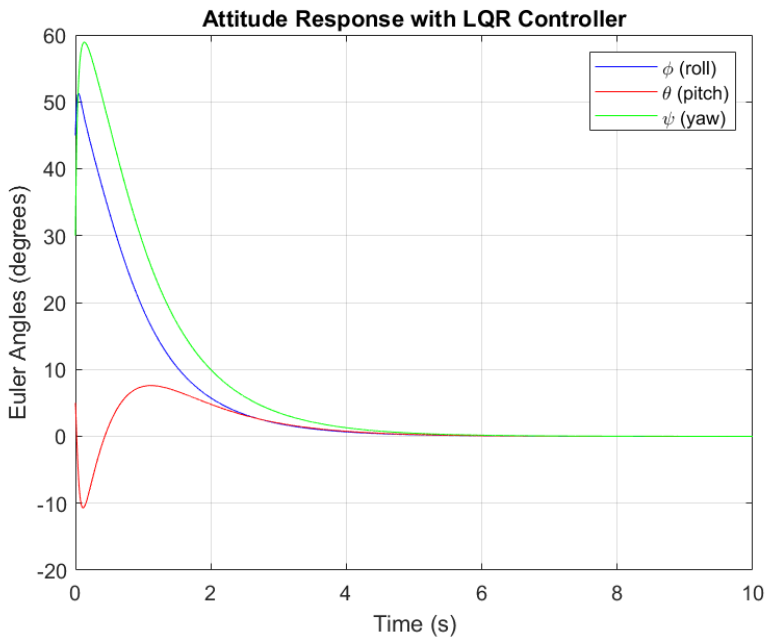

Figure 2. Euler Angle LQR Stabilization

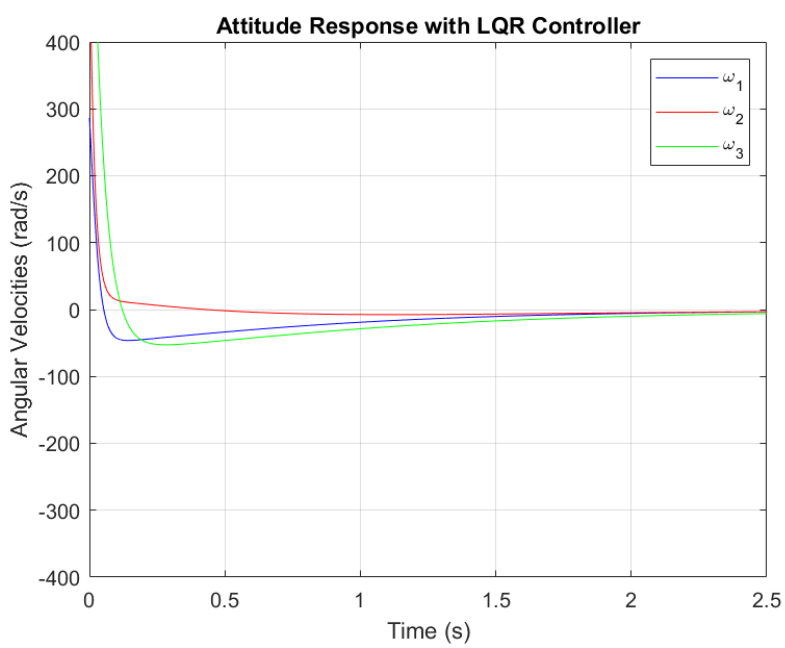

Figure 3. Angular Velocities LQR Stabilization

We can see in both graphs, 2 and 3, how the LQR controller settles the values down to zero. We chose the value of $R = 10$ to provide a less aggressive controller so it would be easier to see how the values change in the plots. With a lower R value, the settling time would decrease, and depending on the value, the peak time also shifts.

3. **Region of attraction via initial-condition sweep in $(\psi,\ \theta)$.**

   (a) **Grid/box of initial conditions.** Consider only roll and pitch initial conditions, with yaw fixed at zero, that is,

   $$\psi(0) = 0, \quad \omega(0) = 0, \quad \phi(0) \in [-85^\circ, 85^\circ], \quad \theta(0) \in [-85^\circ, 85^\circ]. \tag{6}$$

   Generate a uniform grid of $(\phi(0), \theta(0))$ pairs over this box (choose a reasonable resolution, e.g., 5° increments). For each grid point, simulate the nonlinear closed-loop attitude dynamics using your full-state feedback controller.

   (b) **Convergence criterion.** Define an initial condition as converged if, for a chosen simulation horizon T,

   $$\max_{t \in [T-\Delta,\, T]} \left(|\phi(t)|,\ |\theta(t)|\right) \le \varepsilon, \tag{7}$$

   where ε is a small angle tolerance (e.g., $\varepsilon = 2^\circ$) and $\Delta = 0.5\,\mathrm{s}$ is a short terminal window. Otherwise, define it as non-convergent. (You may also label an initial condition as non-convergent if numerical issues occur or if angles grow without bound.)

   (c) **Region of attraction plot.** Create a scatter plot in the $(\phi(0), \theta(0))$ plane, where

   - convergent initial conditions are plotted in green,
   - non-convergent initial conditions are plotted in red.

   Label axes in degrees and include a descriptive caption. This plot is an empirical approximation of the region of attraction for the nonlinear closed-loop system (restricted to $\psi(0) = 0$ and $\omega(0) = 0$).

   (d) **Discussion.** Briefly discuss the observed shape/extent of the convergent region and explain why a controller fails for large initial angles.

Answers a-d:

The region of attraction, shown in 4, is approximately circular with a radius of about 80°, centered at the origin. We chose a much weaker controller for this question because for more aggressive controller values (lower R values and higher Q values) the controller was too strong an caused the system to fully converge everywhere within the grid. We also lowered the final simulation time from 10 seconds to 3 seconds along with lowering the small angle tolerance, ε, to 1°, to further specify which values converged quicker than others. Convergence fails for initial conditions in (ϕ, θ) over $\pm 80°$ from the origin due to breakdown of the small-angle linearization assumptions, significant influence of the neglected nonlinearity at large angles, and the low-gain nature of the conservative LQR design (high R, low Q), which lacks sufficient control authority for large initial errors. More aggressive controllers would expand the region of attraction but would require larger torques.

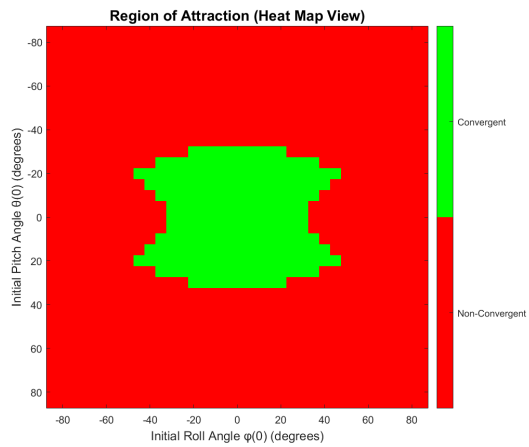

Figure 4. Region of Attraction with R=10000 and Q=0.01*eye(6)

4. **Translational dynamics and position control with integral action.**

The translational dynamics of the multicopter are given by (1), where f is the total thrust magnitude. Note that the thrust direction O_A/De_3 depends on the attitude of the multicopter and is therefore state-dependent and nonlinear.

To simplify the controller design for the translational subsystem, we adopt a commonly used cascaded-control approximation, assuming the attitude dynamics are sufficiently fast relative to the translational dynamics. Under this assumption, the thrust direction can be treated as an independently commandable input through appropriate attitude control.

In particular, consider the idealized translational model

$$m\ddot{p} = mge_3 + \bar{f}, \tag{28}$$

where $\bar{f} \in \mathbb{R}^3$ is a virtual force command generated by the position controller. Note that the idealized model (29) is linear.

(a) Design a full-state feedback controller with integral action for the idealized translational model to achieve position regulation and tracking.

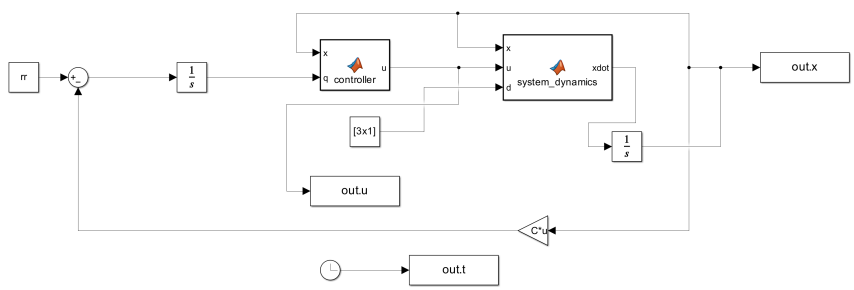

Figure 5. Full-State Feedback Simulink with integral action

(b) Let $\mathbf{p}_{\text{des}} = \mathbf{e}_1$. Simulate the idealized translational dynamics with the controller in closed loop and plot all states.

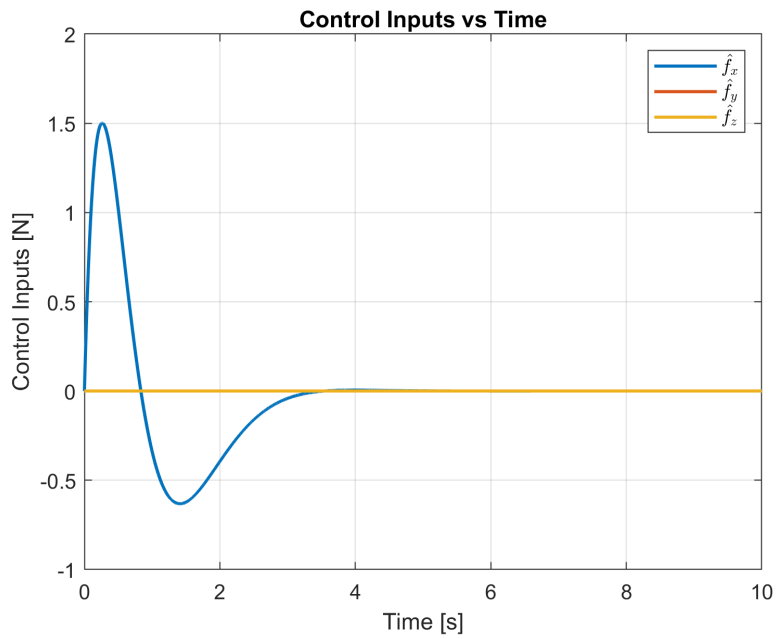

Figure 6. Simulation results. Response of the control input with respect to time. Note that $\hat{f}_z$ is not present in the plot because gravity was subtracted within the controller to maintain hover conditions.

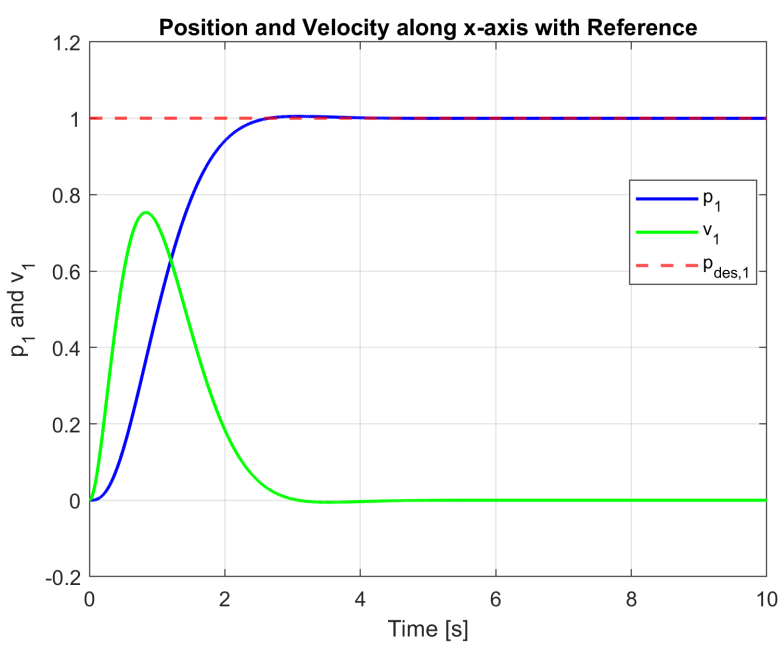

Figure 7. Simulation Results. p_1 converges to the desired point while v_1 conversely reaches 0 once the position condition is met.

(c) Plot the 3D trajectory using plot3 and overlay the virtual force vector $\bar{\mathbf{f}}$ using `quiver3`.

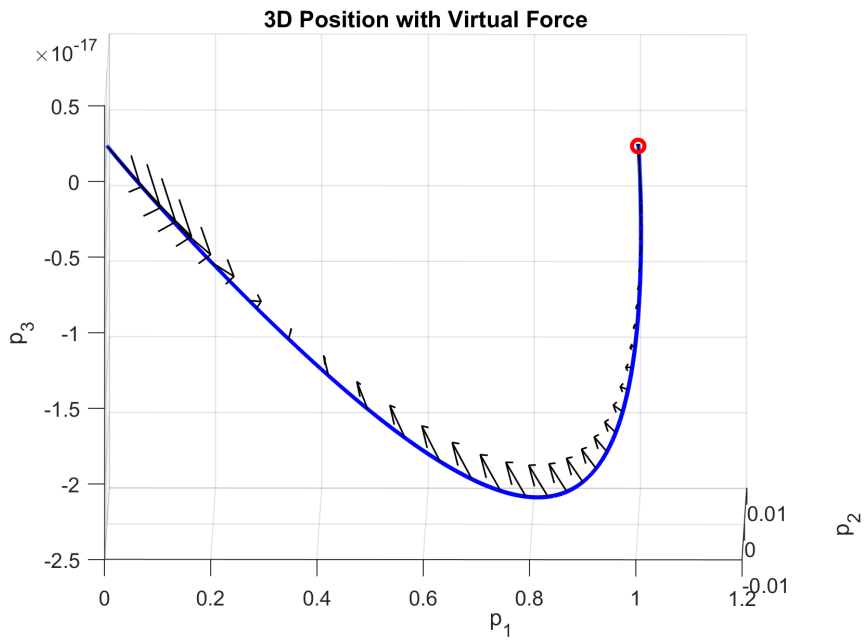

Figure 8. The 3D visualization of the multicopter path. The arrows represent the virtual force vector commanded by the controller.

5. **Combined translational–rotational control (cascaded implementation).**

In this step, you will combine your outer-loop translational controller (virtual force $\bar{\mathbf{f}}$) with your inner-loop attitude controller (torque input $\boldsymbol{\tau}$) to control the full nonlinear multicopter dynamics.

(a) **Virtual-force to attitude/thrust mapping.** At each time t, your translational controller produces a virtual force command $\bar{\mathbf{f}}(t) \in \mathbb{R}^3$ from the idealized model:

$$m\ddot{\mathbf{p}} = mg\mathbf{e}_3 + \bar{\mathbf{f}}. \tag{29}$$

Convert $\bar{\mathbf{f}}(t)$ into a thrust magnitude command $f(t)$ and roll/pitch reference angles $(\phi_{\text{ref}}(t), \theta_{\text{ref}}(t))$ by enforcing the matching condition:

$$OA/D(\phi_{\text{ref}}, \theta_{\text{ref}}, \psi_{\text{ref}})\, \mathbf{e}_3\, f = \bar{\mathbf{f}}. \tag{30}$$

For this project, set $\psi_{\text{ref}}(t) = 0$ for all t. **Answer:** See 5. part(c)

(b) **Inner-loop attitude tracking.** Modify your attitude controller so that it stabilizes the attitude about the time-varying reference $(\phi_{\text{ref}}(t), \theta_{\text{ref}}(t), \psi_{\text{ref}}(t))$ instead of the origin. In particular, define attitude and angular-rate error states, e.g.,

$$e_\phi = \phi - \phi_{\text{ref}}, \quad e_\theta = \theta - \theta_{\text{ref}}, \quad e_\psi = \psi - \psi_{\text{ref}},$$

and the corresponding rate errors, and use your full-state feedback law to generate the torque command:

$$\boldsymbol{\tau}(t) = K_{\text{att}}\, \mathbf{x}_{\text{att,err}}(t). \tag{31}$$

Answer:

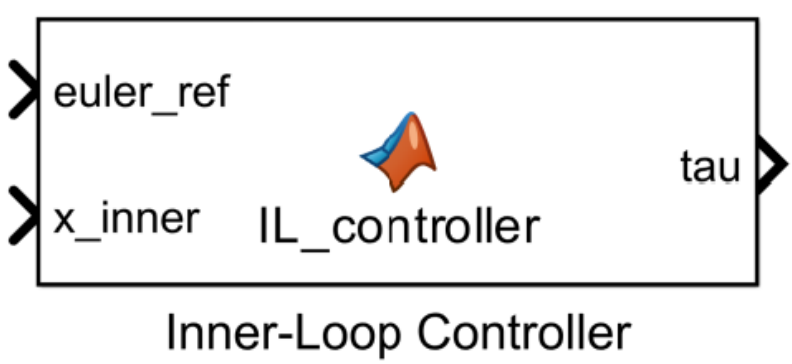

Figure 9. Inner-Loop Controller for Attitude Tracking

```
function tau = IL_controller(euler_ref, x_inner)

    % States
    euler = x_inner(1:3);
    omega = x_inner(4:6);

    % Gains

    tau = -200*(euler - euler_ref) - 50*omega;

end
```

Figure 10. Code for Inner-Loop Controller

The inner-loop controller is responsible for stabilizing the attitude of the multicopter (roll, pitch, yaw) and generating the torques necessary for tracking the desired orientation. $\text{Euler}_{\text{ref}}$ represents the desired orientation provided by the outer-loop controller. The inner-loop state, $\mathbf{x}_{\text{inner}}$, contains the relevant attitude and angular velocity components.

The output, $\boldsymbol{\tau}$, is the corrective torque vector applied along the drone's body axes. The drone uses these inputs to correct its orientation. The implementation is performed using MATLAB functions, where the orientation (Euler) and angular velocity ($\boldsymbol{\omega}$) are handled separately to minimize computational errors.

The torque command is computed as:

$$\boldsymbol{\tau} = -K_p\,(\text{euler} - \text{euler}_{\text{ref}}) - K_d\,\boldsymbol{\omega},$$

where $K_p = 200$ is the proportional gain and $K_d = 50$ is the rotational damping gain. This torque is then sent to the drone's rotational dynamics (see Equation (3)), enabling the multicopter to follow the reference orientation accurately.

(c) **Full nonlinear simulation.** Simulate the full nonlinear multicopter equations of motion (1)–(3). Use your cascaded controller to regulate the position to

$$\mathbf{p}_{\text{des}} = \mathbf{e}_1. \tag{32}$$

Answer:

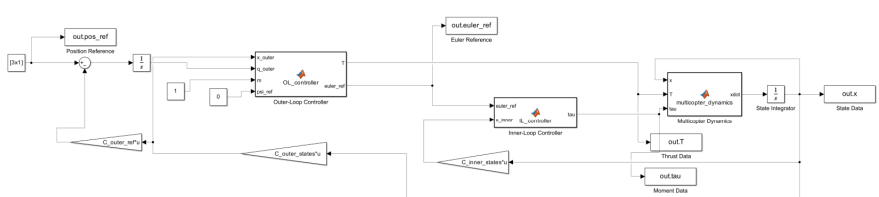

Figure 11. Cascaded Controller for Nonlinear Equation of Motion(EOM) Simulation

(d) **Plots and performance metrics.** Include the following plots:

- $\mathbf{p}(t)$ and $\dot{\mathbf{p}}(t)$ versus time, along with the position tracking error $\mathbf{p}(t) - \mathbf{p}_{\text{des}}$.
- $\phi(t)$ and $\theta(t)$ versus time, together with $\phi_{\text{ref}}(t)$ and $\theta_{\text{ref}}(t)$ on the same plots.
- Thrust command $f(t)$ and torque commands $\tau_1(t), \tau_2(t), \tau_3(t)$ versus time.
- A 3D trajectory plot using `plot3`. Optionally, overlay the commanded thrust direction $OA/D(\phi_{\text{ref}}, \theta_{\text{ref}}, 0)\mathbf{e}_3$ using `quiver3`.

Comment on the transient response, steady-state error, and how well the inner-loop tracks the commanded references.

Answer:

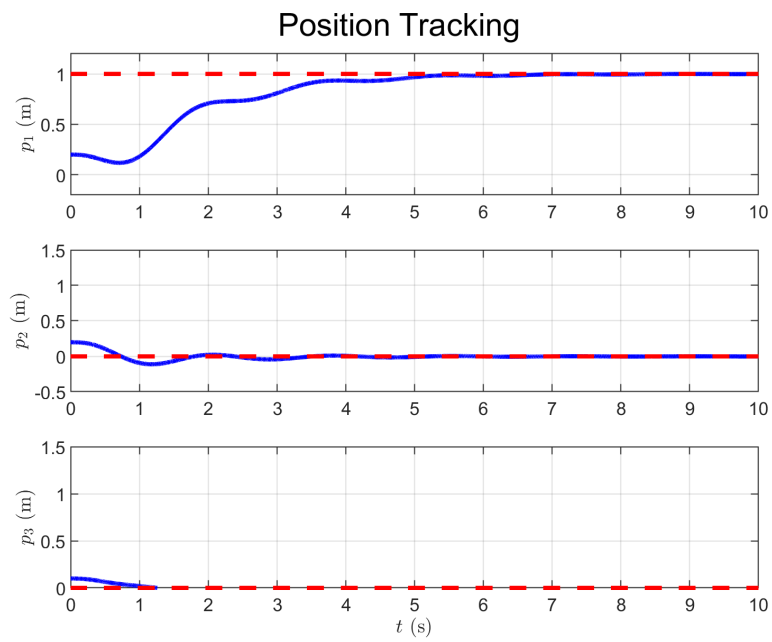

Figure 12. Position Tracking: Shows the drone's actual position versus the desired reference $p_{des} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^T$ over time.

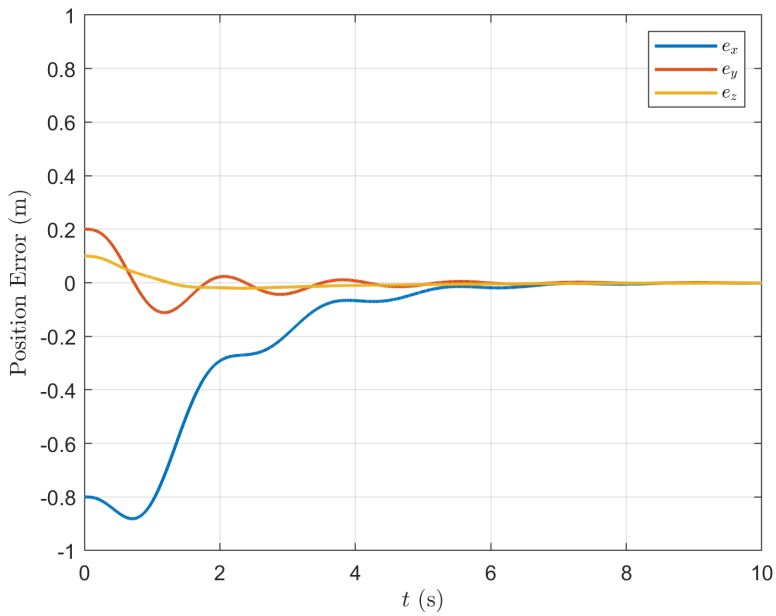

Figure 13. Position Error: Plot of the position tracking errors over time, showing a convergence to zero as the drone approaches the desired point.

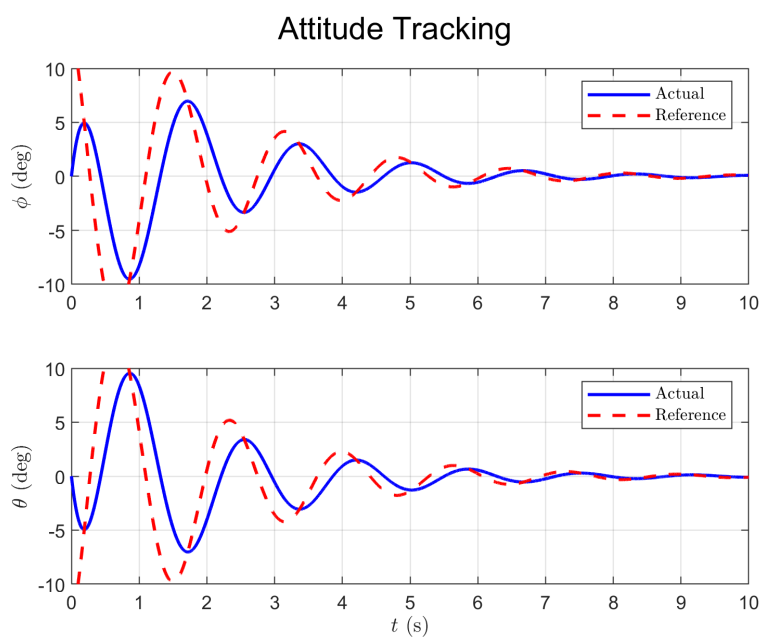

Figure 14. Attitude Tracking: Comparison between the measured roll and pitch angles with their reference values over time. The convergence of the signal indicates attitude stabilization.

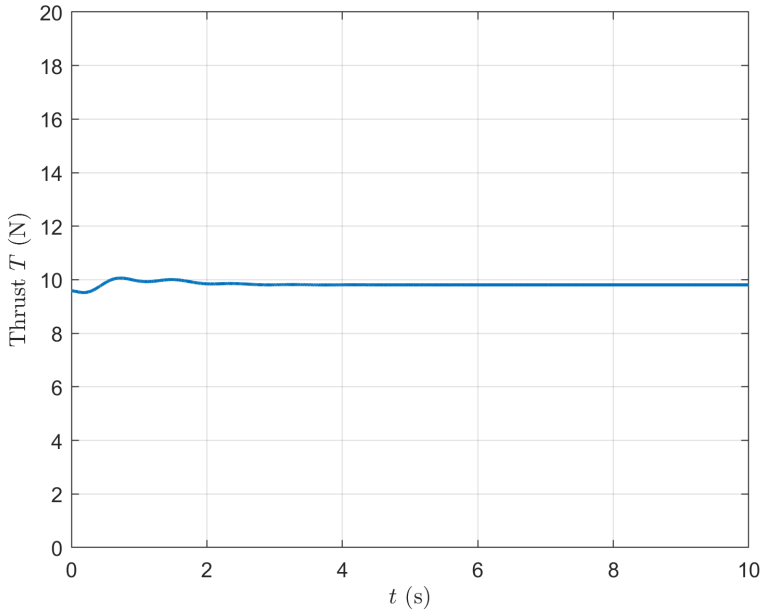

Figure 15. Thrust: Shows the thrust command T applied to the drone over time. T converges to balance gravity and maintain hover equilibrium.

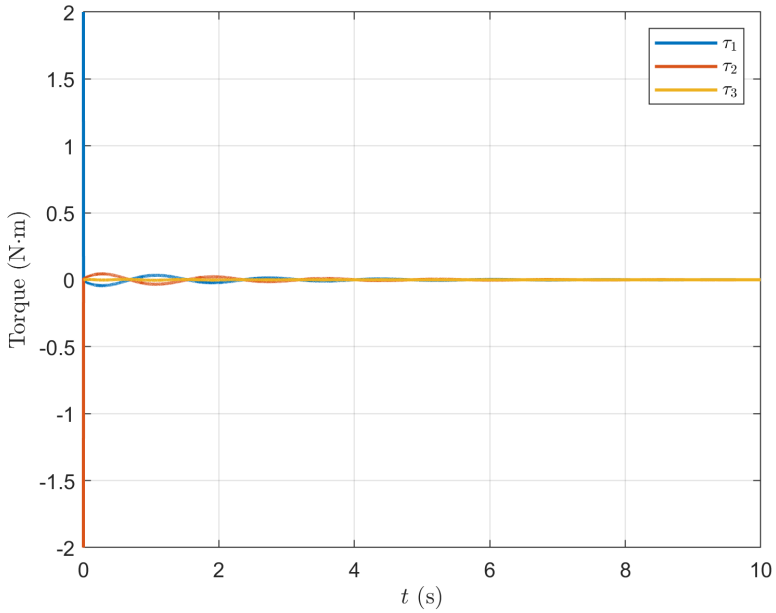

Figure 16. Drone Torques: Displays the torque commands applied to the drone over time for attitude control. Note that torques 1 and 2 correlate with the x and y positional errors observed in Figure 13 and therefore peak higher than torque 3.

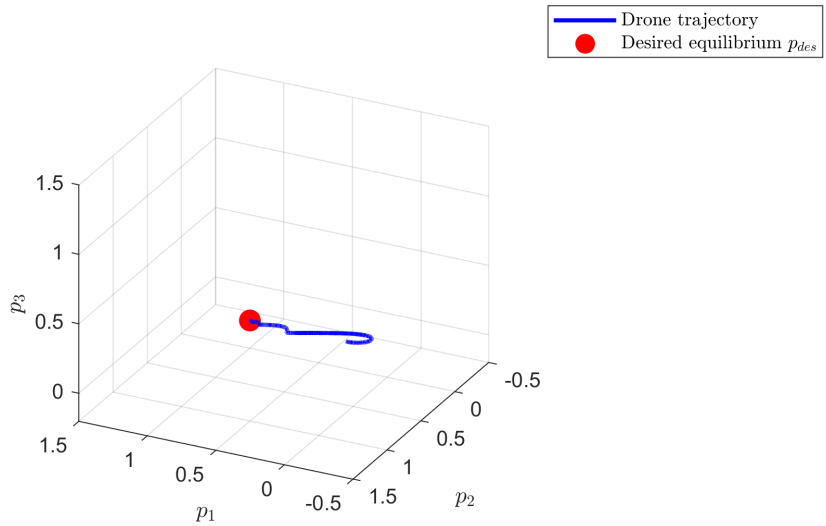

Figure 17. Shows the drone's path in 3D, relative to the desired target p_{des}(the red dot), illustrating the overall position tracking performance.

PERFORMANCE METRICS

- RMS Position Error: 0.3352 m
- Final Position Error: 0.0015 m
- Max Position Error: 0.8824 m

As shown in the plots and in Figure 20, the inner loop tracks the commanded references very well. We first determine this performance using the small Root Mean Square (RMS), Final Position, and Max Position Error. The final position error proves that the final position of the drone and the desired point is minute. Additionally, we can see through visualization that all states converge to their respective references, thus confirming that the performance was effective. Regarding transient response, for position and attitude the drone exhibits small oscillations at the beginning as it accelerates towards the target, with torques and thrust quickly adjusting to stabilize motion. The overshoot is limited, and velocities decay smoothly. Regarding steady-state error, after the transient phase, both position and attitude errors converge to near zero, with the drone maintaining the desired target.

(e) **Stress test (cascaded assumption).** Increase the aggressiveness of the position controller (for example, by assigning faster closed-loop poles or larger gains) and repeat the simulation. Identify when the cascaded assumption begins to degrade performance (e.g., overshoot, oscillations, or loss of tracking).

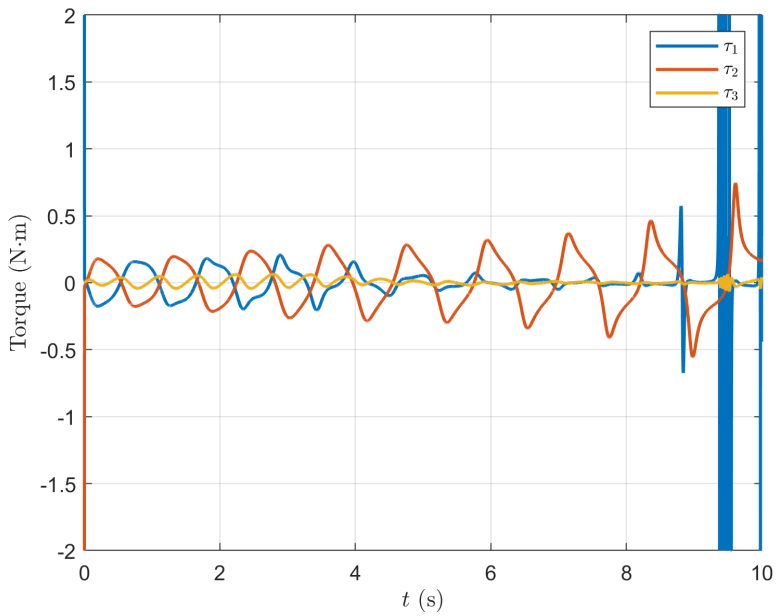

Figure 18. Under the more aggressive controller the torques never converge and demonstrate inconsistent wave forms

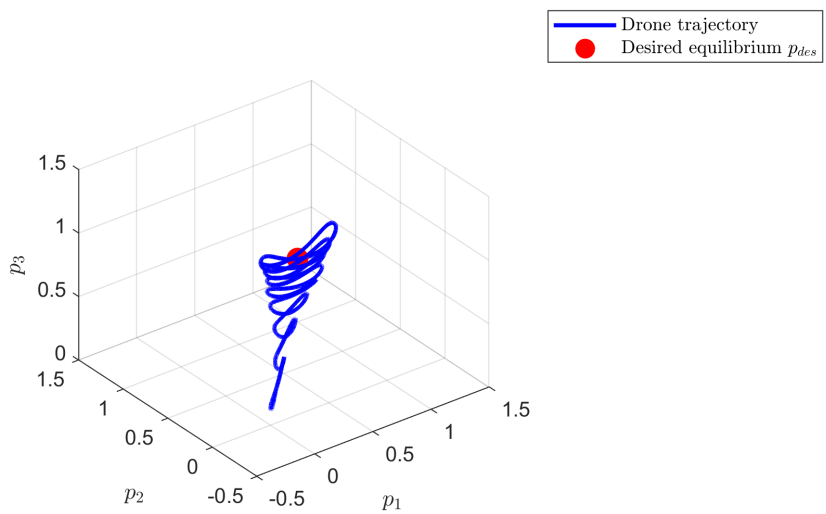

Figure 19. The drone's path in 3D now overshoots the tracking point

When the outer-loop position controller was made more aggressive, the drone significantly overshot the target. This caused high-frequency chattering oscillations in the position and attitude responses. As a result, the cascaded control assumption degraded, leading to a noticeable loss of tracking accuracy.

Credit Author Statement

Christopher Appiah: Question 1 All Derivations and Processes, MATLAB Plotting and Computation for Question 4 and 5
Ethan Hardesty: Question 2 All Derivations and Processes, Simulation,MATLAB Plotting and Computation for Question 2 and 3

Appendix

```
% Performance metrics
rms_error = sqrt(mean(sum(p_err.^2,2)));
final_error = norm(p_err(end,:));
max_error = max(vecnorm(p_err,2,2));
fprintf('\n PERFORMANCE METRICS \n')
fprintf('RMS Position Error: %.4f m\n', rms_error)
fprintf('Final Position Error: %.4f m\n', final_error)
fprintf('Max Position Error: %.4f m\n', max_error)
```

Figure 20. MATLAB code for performance metrics.

Obtaining Desired Pitch and Roll from the Desired Force Vector

Note that

$$\frac{O_A}{D} e_3 = \frac{\mathbf{f}}{\|\mathbf{f}\|} = \hat{\mathbf{f}}. \tag{33}$$

which implies that

$$O_3^T(\psi)\, O_2^T(\theta)\, O_1^T(\phi)\, e_3 = \hat{\mathbf{f}}. \tag{34}$$

Since $\psi(t)$ is user specified, it follows that

$$O_2^T(\theta)\, O_1^T(\phi)\, e_3 = O_3(\psi)\, \hat{\mathbf{f}} = \bar{\mathbf{f}}, \tag{35}$$

which can be expanded to

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix} e_3 = \begin{bmatrix} \bar{f}_1 \\ \bar{f}_2 \\ \bar{f}_3 \end{bmatrix} = \begin{bmatrix} \sin\theta\cos\phi \\ -\sin\phi \\ \cos\theta\cos\phi \end{bmatrix}. \tag{36}$$

This implies that

$$\theta = \operatorname{atan2}(\bar{f}_1,\ \bar{f}_3), \tag{37}$$

$$\phi = \operatorname{atan2}\left(-\bar{f}_2,\ \sqrt{\bar{f}_1^2 + \bar{f}_3^2}\right). \tag{38}$$

Remark Although ϕ can be computed as $\phi = -\sin^{-1}(\bar{f}_2)$, the expression using $\text{atan2}(\cdot,\cdot)$ is preferred in practice. This is because $\sin^{-1}(\cdot)$ returns values only in $[-\pi/2, \pi/2]$ and does not distinguish between configurations with the same sine but different cosine. In contrast, $\text{atan2}(\cdot,\cdot)$ incorporates both sine and cosine information, ensures the correct quadrant of ϕ, and provides improved numerical robustness, particularly when $\cos\phi \approx 0$.